Order Management, Tracking, Cancellation and Returns: final report

Student 7 (Rukhsar Shaikh), branch `feature/orders`. Written 2026-10-24.

1. Task status

Task	Deliverable	Status
ORD-01	Requirements and state diagram (`docs/order-management.md`)	Done
ORD-02	`orders`, `order_items`, `order_status_history`, `returns`, `return_items` tables, migrations `0007` to `0011`	Done
ORD-03	`POST /api/orders` (direct, or from a checkout session), `order_service.create_order`, server-side pricing, confirmation page	Done. Stock deduction waits for the inventory PR (#28).
ORD-04	`update_status()` with transition allow-list and history, `PUT /api/admin/orders/{id}/status`, `GET /api/admin/orders/{id}`	Done
ORD-05	`GET /api/account/orders[/{id}]`, My Orders and tracking pages, login guard	Done
ORD-06	`POST /api/account/orders/{id}/cancel`, refund start	Done. Restock waits for #28.
ORD-07	Returns: request, admin approve or reject, returned, refund; admin routes	Done
ORD-08	Checkpoints 4 and 5 integration testing	Checkpoint 4 test written and passing against Rehan's PR; it runs once #28 is on main. Checkpoint 5 needs Aliza's notifications.
ORD-09	Regression and final docs	Regression recorded in `docs/test-evidence/ORD-09-regression-status.md`; final run after #28 merges.

Tests: 222 backend tests pass, 1 skipped (the checkpoint 4 test, waiting for #28), 70 frontend tests pass, `vite build` succeeds. 18 other backend tests (teammates' login and register tests) fail only on the machine used for testing, because of a `passlib` and `bcrypt` version mismatch. They do not touch orders.

Completed and remaining at a glance

Completed

- Database design and migrations for orders, items, status history, returns and return items.
- Order creation, direct or from a checkout session, with server-side pricing, coupon re-check, single-use session and cart clearing.

- Status transitions with an allow-list and a full history; admin routes to change status and to read one order (customer, address, allowed next steps).
- Customer routes and pages: My Orders, tracking, cancel, return request, order confirmation, all behind the login guard.
- Returns: request, approve or reject, returned, refund; admin routes.
- Cash on Delivery: payment row at confirmation and marked paid on delivery.
- Tests (222 backend, 70 frontend), checkpoint 4 test written, regression record, documentation, this report.

Remaining

- Stock deduct on confirmation and restock on cancel or return on main: waiting for Rehan to merge PR #28. My code and tests are ready.
- Run the checkpoint 4 test and the final regression after #28 merges (ORD-08, ORD-09).
- Order notifications (checkpoint 5): depends on Aliza.
- Tax: the rate is 0 until the team agrees a rule.
- Submit the tasks in the task app and get Anas's review and Rishi's confirmation.

2. The flow of the project (one order, start to finish)

1. **Cart** (Zubiya): the customer adds items. Prices and coupons are checked by the cart and pricing code.
2. **Checkout** (Safiya): the customer picks an address and shipping method. `POST /api/checkout/review` validates everything, and `POST /api/checkout/session` stores a server-side snapshot (address, shipping, items, coupon, totals) with a 30 minute expiry. The browser gets only a `checkout_reference_id`.
3. **Place the order** (Rukhsar): `POST /api/orders` with `{checkout_reference_id, payment_method}`.
    - The session must belong to the customer, be active and unexpired.
    - Prices, shipping cost and the coupon are re-read from the database. If the total changed since checkout the order is refused.
    - The session is marked `completed` in the same transaction, so one session makes one order.
    - The cart is emptied in that transaction too.
4. **Payment** (Aaliya):
    - COD: the order is `CONFIRMED` straight away. A `payments` row (PENDING) is created by my COD hook.
    - Online: the order stays `PLACED` until the backend-verified payment calls `apply_payment_result()`, which confirms it.
5. **Stock** (Rehan): when an order becomes `CONFIRMED`, a status hook deducts stock exactly once. Cancelling a confirmed order, or returning items, restocks.
6. **Fulfilment** (Rishi's admin dashboard): the admin moves the order PROCESSING, PACKED, SHIPPED, OUT_FOR_DELIVERY, DELIVERED with `PUT /api/admin/orders/{id}/status`. Only valid next steps are accepted, and every change is written to `order_status_history` (previous, new, who, when, remarks).
7. **Delivered COD**: the COD hook marks the payment and the order `SUCCESS` (cash collected).
8. **After delivery**: within 7 days the customer can request a return. The admin approves or rejects; an approved return goes RETURNED, then REFUND_PENDING, then REFUNDED. A customer can also

cancel in PLACED, CONFIRMED or PROCESSING; a paid online order moves on to a refund.

```
Normal path:
  PLACED -> CONFIRMED -> PROCESSING -> PACKED -> SHIPPED -> OUT_FOR_DELIVERY -> DELIVERED

Cancel (PLACED, CONFIRMED or PROCESSING):
  -> CANCELLED   (paid online: -> REFUND_PENDING -> REFUNDED)

Return (within 7 days of delivery):
  DELIVERED -> RETURN_REQUESTED -> RETURN_APPROVED -> RETURNED -> REFUND_PENDING -> REFUNDED
  (a rejected return goes back to DELIVERED)
```

3. Key design decisions

- **One door for status changes.** Every status change goes through `order_service.update_status()`. It holds the allow-list, writes the history, and then calls registered hooks. Other modules plug in with `register_status_hook()` instead of editing order code: inventory (stock), COD payment, and later notifications.
- **The server decides prices.** The client never sends a price, total or shipping cost. Order creation rejects extra fields, and everything priced is re-read from the database.
- **One transaction per request.** The routes commit once and roll back on any error, so an order, its history, the stock change, the session and the cart either all happen or none do.
- **Light table definitions for other modules' data.** Products, shipping, addresses, sessions and carts are read through small `table()` definitions, so my code does not clash with the owners' models when branches merge.
- **Role checks on every route.** Customers see only their own orders; admin routes need the admin role.
- **Migrations merged, not rewritten.** `0011_merge_heads` joins the parallel migration branches.

4. What was learned

- **State machines.** An allow-list of transitions is easier to test and safer than scattered `if` checks; the history table gives a free audit trail.
- **Service layer.** Routes stay thin (parse, call, map errors to 404, 409 or 422). Rules live in services that can be tested without HTTP.
- **Never trust the browser** for money. Re-price on the server and compare with what the customer was shown.
- **Transactions and idempotency.** Mark a checkout session used in the same transaction, and make repeated payment callbacks harmless, so retries cannot double-deduct stock or create a second order.
- **Integration contracts.** Teams agree on small, explicit contracts (the `checkout_reference_id`, the hook signature, status values) and write them down. Most integration bugs found here were contract gaps: a COD order with no payments row, a hard-coded checkout item, a session that could be reused.
- **Alembic with many branches.** Parallel migrations create multiple heads; fix them with a merge migration, and never run `upgrade head` on a shared database that may be at a teammate's unmerged revision.
- **Testing.** SQLite in memory plus dependency overrides makes API tests fast. Test the rule that was almost missed (a failed order must not use up the session or empty the cart), not only the happy path.
- **Git in a team.** Merge main often, resolve conflicts by keeping both sides' intent, review the other person's diff before merging, and keep files in UTF-8 (a wrong-encoding save garbled the docs once).

5. Still open

Item	Owner	Note
Stock deduct and restock on main	Rehan, PR #28	Merges cleanly with this branch and passes all tests; needs merging.
Checkpoint 4 test activates	Automatic	`test_checkpoint4_order_payment_inventory.py` runs once #28 is on main.
Order notifications (checkpoint 5)	Aliza	Not started on the order side.
Tax	Team decision	Rate is 0 until agreed.
Real foreign keys for user, address, product, variant, coupon	Later migration	Kept off the ORM so other modules' branches merge first.